

API Gateway - Osztály szintű dokumentáció

April 30, 2026

updating prices. On shutdown, it cancels the background tasks and shuts down the upstream client gracefully.

- **Parameters:** **app** (*FastAPI*) – The FastAPI application instance for which the lifespan is being defined.

1.3 gateway.app.middleware module

1.3.1 class RequestTimingMiddleware(app: Callable[[MutableMapping[str, Any], Callable[[], Awaitable[MutableMapping[str, Any]]], Callable[[MutableMapping[str, Any], Awaitable[None]]], Awaitable[None]], dispatch: Callable[[Request, Callable[[Request], Awaitable[Response]]], Awaitable[Response]] | None = None)

Bases: `BaseHTTPMiddleware`

Middleware to measure the time taken by the gateway to process each request.

***async* dispatch(request: fastapi.Request, call_next: Callable)**
Records the start time of the request processing, calls the next middleware or route handler, and then returns the response.

- **Parameters:**
 - **request** (*Request*) – The incoming FastAPI Request object.
 - **call_next** (*Callable*) – A callable that takes a Request and returns a Response, representing the next step in the middleware chain or the final route handler.

1.3.2 init_middleware(app: fastapi.FastAPI) → None

Initializes the middleware for the FastAPI application, including the Request-TimingMiddleware for measuring request processing time and the CORSMiddleware for handling Cross-Origin Resource Sharing (CORS) with permissive settings.

2 gateway.auth package

2.0.1 *async* validate_api_key(auth: fastapi.security.HTTPAuthorizationCredentials = fastapi.Security) → APIKey

Validates the provided API key from the request headers. It checks for the presence of the API key, verifies its length, checks it against the database for validity and active status, and returns the corresponding APIKey record if valid.

- **Parameters:** `auth` (*HTTPAuthorizationCredentials*) – The *HTTPAuthorizationCredentials* object provided by the HTTPBearer security scheme, containing the API key from the request headers.
- **Returns:** The *APIKey* ORM object corresponding to the valid API key provided in the request.
- **Return type:** *APIKey*

2.1 Submodules

2.2 gateway.auth.keys module

2.2.1 *async* validate_api_key(auth: fastapi.security.HTTPAuthorizationCredentials = fastapi.Security) → APIKey

Validates the provided API key from the request headers. It checks for the presence of the API key, verifies its length, checks it against the database for validity and active status, and returns the corresponding *APIKey* record if valid.

- **Parameters:** `auth` (*HTTPAuthorizationCredentials*) – The *HTTPAuthorizationCredentials* object provided by the HTTPBearer security scheme, containing the API key from the request headers.
- **Returns:** The *APIKey* ORM object corresponding to the valid API key provided in the request.
- **Return type:** *APIKey*

3 gateway.clients package

3.0.1 *class* UpstreamClient(base_url: str, api_key: str)

Bases: `object`

A client for communicating with the upstream LLM provider. It manages the connection and provides methods to send requests to the provider's API.

async **create_chat_completion(payload)** Sends a request to the upstream provider to create a chat completion with the given payload. Handles generating the appropriate request based on whether the request is streaming or not, and includes usage information if streaming for logging.

- **Parameters:** `payload` (*dict*) – A dictionary containing the parameters for the chat completion request, such as `model`, `messages`, `max_tokens`, etc.

async **shutdown()** → `None` Shuts down the upstream client by closing the *AsyncOpenAI* instance and setting it to `None`. This method should be called when the client is no longer needed to clean up resources.

***async startup()* → None** Starts up the upstream client by initializing the AsyncOpenAI instance with the provided base URL and API key. This method should be called before making any requests to the upstream provider.

3.1 Submodules

3.2 gateway.clients.upstream module

3.2.1 *class* UpstreamClient(base_url: str, api_key: str)

Bases: `object`

A client for communicating with the upstream LLM provider. It manages the connection and provides methods to send requests to the provider's API.

async create_chat_completion(payload) Sends a request to the upstream provider to create a chat completion with the given payload. Handles generating the appropriate request based on whether the request is streaming or not, and includes usage information if streaming for logging.

- **Parameters:** **payload** (*dict*) – A dictionary containing the parameters for the chat completion request, such as model, messages, max_tokens, etc.

***async shutdown()* → None** Shuts down the upstream client by closing the AsyncOpenAI instance and setting it to None. This method should be called when the client is no longer needed to clean up resources.

***async startup()* → None** Starts up the upstream client by initializing the AsyncOpenAI instance with the provided base URL and API key. This method should be called before making any requests to the upstream provider.

4 gateway.db package

4.0.1 *async* db_key_check(api_key: str, session=None) → APIKey

Check if the provided API key is valid and associated with an active project. If a session is provided, it uses that session; otherwise, it creates a new session for the query.

- **Parameters:**
 - **api_key** (*str*) – The API key to check.
 - **session** (*optional*) – An optional SQLAlchemy session to use for the database query. If None, a new session will be created.
- **Returns:** The API key record if it is valid and associated with an active project.
- **Return type:** *APIKey*

4.0.2 *async db_limit_change*(api_key: APIKey, request_delta: int = 0, change_by_tokens: int = 0, change_by_price: float = 0, session=None)

Update all relevant quotas for a given API key by the specified values. The function is designed to be atomic by using a transactional session and row-level locks on the relevant quotas.

- **Parameters:**

- **api_key** (*APIKey*) – The APIKey ORM object for which the quotas should be updated.
- **request_delta** (*int* , *optional*) – The change in the number of requests to allocate (positive to allocate, negative to deallocate).
- **change_by_tokens** (*int* , *optional*) – The change in the number of tokens to allocate (positive to allocate, negative to deallocate).
- **change_by_price** (*float* , *optional*) – The change in the price to allocate (positive to allocate, negative to deallocate).
- **session** (*optional*) – An optional SQLAlchemy session to use for the database operations. If None, a new transactional session will be created for this operation.

4.0.3 *async db_limit_check_and_allocation*(api_key: APIKey, estimated_tokens: int, estimated_price: float) → bool

Checks if the quotas for a given API key allow for the estimated tokens/price, if so, allocates them. The function is designed to be atomic by using a transactional session and row-level locks on the relevant quotas.

- **Parameters:**

- **api_key** (*APIKey*) – The APIKey ORM object for which the quotas should be checked and allocated.
- **estimated_tokens** (*int*) – The estimated number of tokens that the request will consume, used for token quota checking.
- **estimated_price** (*float*) – The estimated price of the request, used for price quota checking.

- **Returns:** True if the request is within the quotas and the estimated tokens/price have been allocated, False if any quota would be exceeded.
- **Return type:** *bool*

4.0.4 *async expire_quotas_by_batch*(session=None, batch_size: int = 100)

Expires quotas in the database by updating their status to EXPIRED if the expiration time has passed. This function processes quotas in batches to avoid locking too many rows at once.

- **Parameters:**

- **session** (*optional*) – An optional SQLAlchemy session to use for the database operations. If None, a new transactional session will be created for the operation.
- **batch_size** (*int* , *optional*) – The number of quota records to process in each batch. Default is 100.

- **Returns:** The total number of quotas that were expired.
- **Return type:** *int*

4.0.5 *async* get_quota_count(session=None) → int

Calculate the total number of quota entries in the database. If a session is provided, it uses that session; otherwise, it creates a new transactional session for the query.

- **Parameters:** **session** (*optional*) – An optional SQLAlchemy session to use for the database query. If None, a new session will be created.
- **Returns:** The total number of quota entries in the database.
- **Return type:** *int*

4.0.6 *async* refresh_quotas_by_batch(session=None, batch_size: int = 100)

Refreshes quotas in the database by resetting the allocated amounts and updating next reset time if the next reset time has passed. This function processes quotas in batches to avoid locking too many rows at once.

- **Parameters:**

- **session** (*optional*) – An optional SQLAlchemy session to use for the database operations. If None, a new transactional session will be created for the operation.
- **batch_size** (*int* , *optional*) – The number of quota records to process in each batch. Default is 100.

- **Returns:** The total number of quotas that were refreshed.
- **Return type:** *int*

4.1 Submodules

4.2 gateway.db.helpers module

4.2.1 *async* get_quota_count(session=None) → int

Calculate the total number of quota entries in the database. If a session is provided, it uses that session; otherwise, it creates a new transactional session for the query.

- **Parameters:** `session` (*optional*) – An optional SQLAlchemy session to use for the database query. If None, a new session will be created.
- **Returns:** The total number of quota entries in the database.
- **Return type:** *int*

4.3 gateway.db.keys module

4.3.1 `async db_key_check(api_key: str, session=None) → APIKey`

Check if the provided API key is valid and associated with an active project. If a session is provided, it uses that session; otherwise, it creates a new session for the query.

- **Parameters:**
 - `api_key` (*str*) – The API key to check.
 - `session` (*optional*) – An optional SQLAlchemy session to use for the database query. If None, a new session will be created.
- **Returns:** The API key record if it is valid and associated with an active project.
- **Return type:** *APIKey*

4.4 gateway.db.limits module

4.4.1 `async db_get_quotas_by_key(api_key: APIKey, session=None) → Tuple[list[Quota], list[Quota], list[Quota]]`

Returns a tuple of the different quota types (request, token, price) for a given key. The function is designed to be atomic by using a transactional session and row-level locks on the relevant quotas.

- **Parameters:**
 - `api_key` (*APIKey*) – The APIKey ORM object for which the quotas should be retrieved.
 - `session` (*optional*) – An optional SQLAlchemy session to use for the database operations. If None, a new transactional session will be created for this operation.
- **Returns:** A tuple containing three lists of Quota objects: (request_quotas, token_quotas, price_quotas) that are relevant to the given API key.
- **Return type:** *Tuple[list[Quota], list[Quota], list[Quota]]*

4.4.2 *async db_limit_change*(api_key: APIKey, request_delta: int = 0, change_by_tokens: int = 0, change_by_price: float = 0, session=None)

Update all relevant quotas for a given API key by the specified values. The function is designed to be atomic by using a transactional session and row-level locks on the relevant quotas.

- **Parameters:**

- **api_key** (*APIKey*) – The APIKey ORM object for which the quotas should be updated.
- **request_delta** (*int* , *optional*) – The change in the number of requests to allocate (positive to allocate, negative to deallocate).
- **change_by_tokens** (*int* , *optional*) – The change in the number of tokens to allocate (positive to allocate, negative to deallocate).
- **change_by_price** (*float* , *optional*) – The change in the price to allocate (positive to allocate, negative to deallocate).
- **session** (*optional*) – An optional SQLAlchemy session to use for the database operations. If None, a new transactional session will be created for this operation.

4.4.3 *async db_limit_check_and_allocation*(api_key: APIKey, estimated_tokens: int, estimated_price: float) → bool

Checks if the quotas for a given API key allow for the estimated tokens/price, if so, allocates them. The function is designed to be atomic by using a transactional session and row-level locks on the relevant quotas.

- **Parameters:**

- **api_key** (*APIKey*) – The APIKey ORM object for which the quotas should be checked and allocated.
- **estimated_tokens** (*int*) – The estimated number of tokens that the request will consume, used for token quota checking.
- **estimated_price** (*float*) – The estimated price of the request, used for price quota checking.

- **Returns:** True if the request is within the quotas and the estimated tokens/price have been allocated, False if any quota would be exceeded.
- **Return type:** *bool*

4.5 gateway.db.refresh module

4.5.1 *async expire_quotas_by_batch*(session=None, batch_size: int = 100)

Expires quotas in the database by updating their status to EXPIRED if the expiration time has passed. This function processes quotas in batches to avoid locking too many rows at once.

- **Parameters:**

- **session** (*optional*) – An optional SQLAlchemy session to use for the database operations. If None, a new transactional session will be created for the operation.
- **batch_size** (*int* , *optional*) – The number of quota records to process in each batch. Default is 100.

- **Returns:** The total number of quotas that were expired.
- **Return type:** *int*

4.5.2 `async refresh_quotas_by_batch(session=None, batch_size: int = 100)`

Refreshes quotas in the database by resetting the allocated amounts and updating next reset time if the next reset time has passed. This function processes quotas in batches to avoid locking too many rows at once.

- **Parameters:**

- **session** (*optional*) – An optional SQLAlchemy session to use for the database operations. If None, a new transactional session will be created for the operation.
- **batch_size** (*int* , *optional*) – The number of quota records to process in each batch. Default is 100.

- **Returns:** The total number of quotas that were refreshed.
- **Return type:** *int*

5 gateway.limits package

5.0.1 `async check_limits_costs(body: OpenAIRequest, key_info: APIKey = fastapi.Depends) → ValidatedRequest`

Check if the request is expected to exceed the user's quota limits based on token and cost estimates. If the request is within limits, it allocates the estimated tokens and cost against the user's quota.

- **Parameters:**

- **body** (*OpenAIRequest*) – The OpenAIRequest DTO containing the details of the incoming request.
- **key_info** (*APIKey*) – The APIKey ORM object for the authenticated API key, injected by the `validate_api_key` dependency.

- **Returns:** A DTO containing the validated request information along with the estimated tokens and cost.
- **Return type:** *ValidatedRequest*

5.1 Submodules

5.2 gateway.limits.quota module

5.2.1 *async* `check_limits_costs(body: OpenAIRequest, key_info: APIKey = fastapi.Depends) → ValidatedRequest`

Check if the request is expected to exceed the user's quota limits based on token and cost estimates. If the request is within limits, it allocates the estimated tokens and cost against the user's quota.

- **Parameters:**
 - **body** (*OpenAIRequest*) – The OpenAIRequest DTO containing the details of the incoming request.
 - **key_info** (*APIKey*) – The APIKey ORM object for the authenticated API key, injected by the `validate_api_key` dependency.
- **Returns:** A DTO containing the validated request information along with the estimated tokens and cost.
- **Return type:** *ValidatedRequest*

6 gateway.logging package

6.0.1 *async* `prepare_usage_entry(request: fastapi.Request, validated_request: ValidatedRequest, upstream_latency, gateway_overhead, ttft, total_time, chat_completion=None, status_code=200, failed_upstream=False)`

Extract the relevant usage information from the completion response and calls the logger to store it in the database.

- **Parameters:**
 - **request** (*Request*) – The original FastAPI request object.
 - **validated_request** (*ValidatedRequest*) – The validated request DTO containing estimates and user/project/key information.
 - **upstream_latency** – Time taken for the upstream provider to respond.
 - **gateway_overhead** – Time taken by the gateway to process the request excluding upstream latency.
 - **ttft** – Time to first token, if applicable.
 - **total_time** – Total time taken for the entire request processing.
 - **chat_completion** – The response object from the chat completion API, None if the request failed before receiving a response.
 - **status_code** – The HTTP status code of the response, defaulting to 200 for successful requests.
 - **failed_upstream** – Boolean indicating if the failure was due to an upstream provider issue.

6.0.2 *async* usage_logger(entry: UsageLogEntry, failed_upstream: bool = False)

Logs the usage information to the database and updates the API key limits accordingly.

- **Parameters:**

- **entry** (*UsageLogEntry*) – The UsageLogEntry DTO containing all the relevant usage information to be logged.
- **failed_upstream** (*bool* , *optional*) – Boolean indicating if the failure was due to an upstream provider issue, which affects how limits are adjusted.

6.1 Submodules

6.2 gateway.logging.extractor module

6.2.1 *async* prepare_usage_entry(request: fastapi.Request, validated_request: ValidatedRequest, upstream_latency, gateway_overhead, ttft, total_time, chat_completion=None, status_code=200, failed_upstream=False)

Extract the relevant usage information from the completion response and calls the logger to store it in the database.

- **Parameters:**

- **request** (*Request*) – The original FastAPI request object.
- **validated_request** (*ValidatedRequest*) – The validated request DTO containing estimates and user/project/key information.
- **upstream_latency** – Time taken for the upstream provider to respond.
- **gateway_overhead** – Time taken by the gateway to process the request excluding upstream latency.
- **ttft** – Time to first token, if applicable.
- **total_time** – Total time taken for the entire request processing.
- **chat_completion** – The response object from the chat completion API, None if the request failed before receiving a response.
- **status_code** – The HTTP status code of the response, defaulting to 200 for successful requests.
- **failed_upstream** – Boolean indicating if the failure was due to an upstream provider issue.

6.3 gateway.logging.metadata module

6.3.1 *async* usage_logger(entry: UsageLogEntry, failed_upstream: bool = False)

Logs the usage information to the database and updates the API key limits accordingly.

- **Parameters:**

- **entry** (*UsageLogEntry*) – The UsageLogEntry DTO containing all the relevant usage information to be logged.
- **failed_upstream** (*bool* , *optional*) – Boolean indicating if the failure was due to an upstream provider issue, which affects how limits are adjusted.

7 gateway package

7.1 Subpackages

- gateway.app package
 - create_app()
 - Submodules
 - gateway.app.factory module
 - * create_app()
 - * lifespan()
 - gateway.app.middleware module
 - * RequestTimingMiddleware
 - RequestTimingMiddleware.dispatch()
 - * init_middleware()
- gateway.auth package
 - validate_api_key()
 - Submodules
 - gateway.auth.keys module
 - * validate_api_key()
- gateway.clients package
 - UpstreamClient
 - * UpstreamClient.create_chat_completion()
 - * UpstreamClient.shutdown()
 - * UpstreamClient.startup()
 - Submodules
 - gateway.clients.upstream module
 - * UpstreamClient

- UpstreamClient.create_chat_completion()
 - UpstreamClient.shutdown()
 - UpstreamClient.startup()
- gateway.db package
 - db_key_check()
 - db_limit_change()
 - db_limit_check_and_allocation()
 - expire_quotas_by_batch()
 - get_quota_count()
 - refresh_quotas_by_batch()
 - Submodules
 - gateway.db.helpers module
 - * get_quota_count()
 - gateway.db.keys module
 - * db_key_check()
 - gateway.db.limits module
 - * db_get_quotas_by_key()
 - * db_limit_change()
 - * db_limit_check_and_allocation()
 - gateway.db.refresh module
 - * expire_quotas_by_batch()
 - * refresh_quotas_by_batch()
- gateway.limits package
 - check_limits_costs()
 - Submodules
 - gateway.limits.quota module
 - * check_limits_costs()
- gateway.logging package
 - prepare_usage_entry()
 - usage_logger()
 - Submodules
 - gateway.logging.extractor module
 - * prepare_usage_entry()
 - gateway.logging.metadata module
 - * usage_logger()
- gateway.models package
 - OpenAIMessage
 - * OpenAIMessage.content
 - * OpenAIMessage.role

- OpenAIRequest
 - * OpenAIRequest.messages
 - * OpenAIRequest.model
- UsageLogEntry
 - * UsageLogEntry.completion_tokens
 - * UsageLogEntry.estimated_tokens
 - * UsageLogEntry.finish_reason
 - * UsageLogEntry.gateway_overhead
 - * UsageLogEntry.internal_cost_estimate
 - * UsageLogEntry.internal_cost_final
 - * UsageLogEntry.is_streaming
 - * UsageLogEntry.key_id
 - * UsageLogEntry.model
 - * UsageLogEntry.project_id
 - * UsageLogEntry.prompt_tokens
 - * UsageLogEntry.request_id
 - * UsageLogEntry.request_type
 - * UsageLogEntry.status_code
 - * UsageLogEntry.temperature
 - * UsageLogEntry.timestamp
 - * UsageLogEntry.top_k
 - * UsageLogEntry.top_p
 - * UsageLogEntry.total_latency
 - * UsageLogEntry.total_tokens
 - * UsageLogEntry.ttft
 - * UsageLogEntry.upstream_latency
 - * UsageLogEntry.user_id
- ValidatedRequest
 - * ValidatedRequest.body
 - * ValidatedRequest.estimated_tokens
 - * ValidatedRequest.internal_cost_estimate
 - * ValidatedRequest.key_id
 - * ValidatedRequest.project_id
 - * ValidatedRequest.user_id
- Subpackages
 - * gateway.models.dto_models package
 - OpenAIMessage
 - OpenAIRequest
 - UsageLogEntry
 - ValidatedRequest
 - Submodules
 - gateway.models.dto_models.openai module
 - gateway.models.dto_models.quotas module
 - gateway.models.dto_models.usage module

- gateway.routes package
 - Submodules
 - gateway.routes.chat module
 - * `forward_request()`
 - gateway.routes.health module
 - * `health()`
 - * `root()`
- gateway.utils package
 - `calculate_cost()`
 - `get_token_count()`
 - `stream_generator()`
 - Submodules
 - gateway.utils.price module
 - * `calculate_cost()`
 - gateway.utils.streaming module
 - * `stream_generator()`
 - gateway.utils.usage module
 - * `get_token_count()`

8 gateway.models.dto_models package

8.0.1 `class OpenAIMessage(*args: Any, **kwargs: Any)`

Bases: `BaseModel`

A class representing a message in the OpenAI chat format.

content : `str` / `List[Dict[str, Any]]` = *Ellipsis*

role : `str` = *Ellipsis*

8.0.2 `class OpenAIRequest(*args: Any, **kwargs: Any)`

Bases: `BaseModel`

A class representing a request to the OpenAI chat API.

messages : `List[OpenAIMessage]` = *Ellipsis*

model : `str` = *Ellipsis*

8.0.3 `class UsageLogEntry(*args: Any, **kwargs: Any)`

Bases: `BaseModel`

A class representing an entry in the usage log. Shares the fields with the `UsageEntry` model used for database storage.

SEE ALSO

UsageEntry The model used for storing usage data in the database.

completion_tokens : *int* / *None*

estimated_tokens : *int* / *None*

finish_reason : *str* / *None*

gateway_overhead : *float* / *None*

internal_cost_estimate : *float* / *None*

internal_cost_final : *float* / *None*

is_streaming : *bool* / *None*

key_id : *int*

model : *str* / *None*

project_id : *int*

prompt_tokens : *int* / *None*

request_id : *str* / *None*

request_type : *str* / *None*

status_code : *int* / *None*

temperature : *float* / *None*

timestamp : *datetime*

top_k : *int* / *None*

top_p : *float* / *None*

total_latency : *float* / *None*

total_tokens : *int* / *None*

ttft : float / None

upstream_latency : float / None

user_id : int

8.0.4 *class* ValidatedRequest(*args: Any, **kwargs: Any)

Bases: BaseModel

A class representing a request made to the gateway that has been validated for authentication, authorization and limits.

body : OpenAIRequest

estimated_tokens : int

internal_cost_estimate : float / None

key_id : int

project_id : int

user_id : int

8.1 Submodules

8.2 gateway.models.dto_models.openai module

8.2.1 *class* OpenAIMessage(*args: Any, **kwargs: Any)

Bases: BaseModel

A class representing a message in the OpenAI chat format.

content : str / List[Dict[str, Any]] = Ellipsis

role : str = Ellipsis

8.2.2 *class* OpenAIRequest(*args: Any, **kwargs: Any)

Bases: BaseModel

A class representing a request to the OpenAI chat API.

messages : List[OpenAIMessage] = Ellipsis

model : str = Ellipsis

8.3 gateway.models.dto_models.quotas module

8.3.1 *class* ValidatedRequest(*args: Any, **kwargs: Any)

Bases: BaseModel

A class representing a request made to the gateway that has been validated for authentication, authorization and limits.

body : OpenAIRequest

estimated_tokens : int

internal_cost_estimate : float / None

key_id : int

project_id : int

user_id : int

8.4 gateway.models.dto_models.usage module

8.4.1 *class* UsageLogEntry(*args: Any, **kwargs: Any)

Bases: BaseModel

A class representing an entry in the usage log. Shares the fields with the UsageEntry model used for database storage.

SEE ALSO

UsageEntry The model used for storing usage data in the database.

completion_tokens : int / None

estimated_tokens : int / None

finish_reason : str / None

gateway_overhead : float / None

internal_cost_estimate : float / None

internal_cost_final : float / None

is_streaming : bool / None

`key_id : int`
`model : str / None`
`project_id : int`
`prompt_tokens : int / None`
`request_id : str / None`
`request_type : str / None`
`status_code : int / None`
`temperature : float / None`
`timestamp : datetime`
`top_k : int / None`
`top_p : float / None`
`total_latency : float / None`
`total_tokens : int / None`
`ttft : float / None`
`upstream_latency : float / None`
`user_id : int`

9 gateway.models package

9.0.1 `class OpenAIMessage(*args: Any, **kwargs: Any)`

Bases: `BaseModel`

A class representing a message in the OpenAI chat format.

`content : str / List[Dict[str, Any]] = Ellipsis`

`role : str = Ellipsis`

9.0.2 *class* OpenAIRequest(*args: Any, **kwargs: Any)

Bases: BaseModel

A class representing a request to the OpenAI chat API.

messages : List[OpenAIMessage] = Ellipsis

model : str = Ellipsis

9.0.3 *class* UsageLogEntry(*args: Any, **kwargs: Any)

Bases: BaseModel

A class representing an entry in the usage log. Shares the fields with the UsageEntry model used for database storage.

SEE ALSO

UsageEntry The model used for storing usage data in the database.

completion_tokens : int / None

estimated_tokens : int / None

finish_reason : str / None

gateway_overhead : float / None

internal_cost_estimate : float / None

internal_cost_final : float / None

is_streaming : bool / None

key_id : int

model : str / None

project_id : int

prompt_tokens : int / None

request_id : str / None

request_type : str / None

`status_code : int / None`

`temperature : float / None`

`timestamp : datetime`

`top_k : int / None`

`top_p : float / None`

`total_latency : float / None`

`total_tokens : int / None`

`ttft : float / None`

`upstream_latency : float / None`

`user_id : int`

9.0.4 `class ValidatedRequest(*args: Any, **kwargs: Any)`

Bases: `BaseModel`

A class representing a request made to the gateway that has been validated for authentication, authorization and limits.

`body : OpenAIRequest`

`estimated_tokens : int`

`internal_cost_estimate : float / None`

`key_id : int`

`project_id : int`

`user_id : int`

9.1 Subpackages

- gateway.models.dto_models package
 - OpenAIMessage
 - * OpenAIMessage.content
 - * OpenAIMessage.role
 - OpenAIRequest
 - * OpenAIRequest.messages
 - * OpenAIRequest.model
 - UsageLogEntry
 - * UsageLogEntry.completion_tokens
 - * UsageLogEntry.estimated_tokens
 - * UsageLogEntry.finish_reason
 - * UsageLogEntry.gateway_overhead
 - * UsageLogEntry.internal_cost_estimate
 - * UsageLogEntry.internal_cost_final
 - * UsageLogEntry.is_streaming
 - * UsageLogEntry.key_id
 - * UsageLogEntry.model
 - * UsageLogEntry.project_id
 - * UsageLogEntry.prompt_tokens
 - * UsageLogEntry.request_id
 - * UsageLogEntry.request_type
 - * UsageLogEntry.status_code
 - * UsageLogEntry.temperature
 - * UsageLogEntry.timestamp
 - * UsageLogEntry.top_k
 - * UsageLogEntry.top_p
 - * UsageLogEntry.total_latency
 - * UsageLogEntry.total_tokens
 - * UsageLogEntry.ttft
 - * UsageLogEntry.upstream_latency
 - * UsageLogEntry.user_id
 - ValidatedRequest
 - * ValidatedRequest.body
 - * ValidatedRequest.estimated_tokens
 - * ValidatedRequest.internal_cost_estimate
 - * ValidatedRequest.key_id
 - * ValidatedRequest.project_id
 - * ValidatedRequest.user_id
 - Submodules
 - gateway.models.dto_models.openai module
 - * OpenAIMessage
 - OpenAIMessage.content

- OpenAIMessage.role
- * OpenAIRequest
 - OpenAIRequest.messages
 - OpenAIRequest.model
- gateway.models.dto_models.quotas module
 - * ValidatedRequest
 - ValidatedRequest.body
 - ValidatedRequest.estimated_tokens
 - ValidatedRequest.internal_cost_estimate
 - ValidatedRequest.key_id
 - ValidatedRequest.project_id
 - ValidatedRequest.user_id
- gateway.models.dto_models.usage module
 - * UsageLogEntry
 - UsageLogEntry.completion_tokens
 - UsageLogEntry.estimated_tokens
 - UsageLogEntry.finish_reason
 - UsageLogEntry.gateway_overhead
 - UsageLogEntry.internal_cost_estimate
 - UsageLogEntry.internal_cost_final
 - UsageLogEntry.is_streaming
 - UsageLogEntry.key_id
 - UsageLogEntry.model
 - UsageLogEntry.project_id
 - UsageLogEntry.prompt_tokens
 - UsageLogEntry.request_id
 - UsageLogEntry.request_type
 - UsageLogEntry.status_code
 - UsageLogEntry.temperature
 - UsageLogEntry.timestamp
 - UsageLogEntry.top_k
 - UsageLogEntry.top_p
 - UsageLogEntry.total_latency
 - UsageLogEntry.total_tokens
 - UsageLogEntry.ttft
 - UsageLogEntry.upstream_latency
 - UsageLogEntry.user_id

10 gateway.routes package

10.1 Submodules

10.2 gateway.routes.chat module

10.2.1 *async* forward_request(request: fastapi.Request, validated_request: ValidatedRequest = fastapi.Depends)

Endpoint to handle chat completion requests.

This function receives an OpenAI compatible chat completion requests, validates the key and enforces the set quotas and limits, forwards the request to the upstream LLM service, and returns the response to the client. It also handles streaming responses and logs usage data for each request.

- **Parameters:**

- **request** (*Request*) – The FastAPI request object, which contains information about the incoming request and the application state.
- **validated_request** (*ValidatedRequest*) – A DTO containing the validated request data, including authentication, authorization and estimated usage information. This is provided by the `check_limits_costs` dependency, which also enforces quotas and limits.

- **Returns:** If the request is not a streaming request, it returns the response from the upstream LLM service directly to the client. If the request is a streaming request, it returns a `StreamingResponse` that streams the response from the upstream LLM service to the client as it is generated, using server-sent events (SSE).

- **Return type:** `StreamingResponse` or any

10.3 gateway.routes.health module

10.3.1 *async* health(request: fastapi.Request)

Endpoint to check the health of the gateway.

- **Returns:** A simple JSON response indicating OK status and optionally the version of the gateway.
- **Return type:** *dict*

10.3.2 *async* root(request: fastapi.Request)

Alias for the health check endpoint, allowing the gateway status to be checked by sending a request to the root path.

11 gateway.utils package

11.0.1 `calculate_cost(usage: Usage, model_ref: str, provider_id: str) → float`

Calculate the estimated cost of a request.

This function attempts to calculate the cost using the Pydantic `genai_prices` library. If it encounters any issues (e.g., missing model information, API errors), it falls back to a default pricing strategy based on token count and default values.

- **Parameters:**

- **usage** (*Usage*) – A `genai_prices.Usage` object containing the input and output token counts.
- **model_ref** (*str*) – A string reference to the model being used (e.g., “zai-org/GLM-4.5-Air-FP8”).
- **provider_id** (*str*) – A string identifying the provider (e.g., “zai-org”).

- **Returns:** A float representing the estimated cost of the request in USD.
- **Return type:** *float*

11.0.2 `get_token_count(messages: List[OpenAIMessage]) → int`

Calculate the total number of tokens in a list of `OpenAIMessage` objects.

This function uses the `tiktoken` library to encode the content of the messages and counts the total number of tokens.

- **Parameters:** **messages** (*List [OpenAIMessage]*) – A list of `OpenAIMessage` DTOs, where each message contains a ‘content’ field that can be a string or an array of content-part dictionaries.
- **Returns:** An integer representing the total number of tokens in the messages.
- **Return type:** *int*

11.0.3 `async stream_generator(raw_stream: openai.AsyncStream.openai.types.chat.ChatCompletionChunk, request: fastapi.Request, validated_request: ValidatedRequest)`

Convert an `AsyncStream` of `ChatCompletionChunk` objects into a Server-Sent Events (SSE) stream.

This function processes each chunk from the upstream stream, extracts relevant metadata, and yields it in a format suitable for SSE. It also handles errors gracefully by yielding an error message in the SSE format if any exceptions occur during streaming.

- **Parameters:**

- **raw_stream** (*AsyncStream* *[**ChatCompletionChunk* *]*) – An AsyncStream of ChatCompletionChunk objects from the OpenAI package.
- **request** (*Request*) – The FastAPI request object.
- **validated_request** (*ValidatedRequest*) – The validated request DTO containing information about the request for logging and usage tracking purposes.
- **Yields:** *str* – A string formatted as an SSE event, containing either the chunk data or an error message.

11.1 Submodules

11.2 gateway.utils.price module

11.2.1 calculate_cost(usage: Usage, model_ref: str, provider_id: str) → float

Calculate the estimated cost of a request.

This function attempts to calculate the cost using the Pydantic genai_prices library. If it encounters any issues (e.g., missing model information, API errors), it falls back to a default pricing strategy based on token count and default values.

- **Parameters:**
 - **usage** (*Usage*) – A genai_prices.Usage object containing the input and output token counts.
 - **model_ref** (*str*) – A string reference to the model being used (e.g., “zai-org/GLM-4.5-Air-FP8”).
 - **provider_id** (*str*) – A string identifying the provider (e.g., “zai-org”).
- **Returns:** A float representing the estimated cost of the request in USD.
- **Return type:** *float*

11.3 gateway.utils.streaming module

11.3.1 async stream_generator(raw_stream: openai.AsyncStream.openai.types.chat.ChatCompletionChunk, request: fastapi.Request, validated_request: ValidatedRequest)

Convert an AsyncStream of ChatCompletionChunk objects into a Server-Sent Events (SSE) stream.

This function processes each chunk from the upstream stream, extracts relevant metadata, and yields it in a format suitable for SSE. It also handles errors gracefully by yielding an error message in the SSE format if any exceptions occur during streaming.

- **Parameters:**

- **raw_stream** (*AsyncStream* [***ChatCompletionChunk** *]) – An *AsyncStream* of *ChatCompletionChunk* objects from the *OpenAI* package.
- **request** (*Request*) – The FastAPI request object.
- **validated_request** (*ValidatedRequest*) – The validated request DTO containing information about the request for logging and usage tracking purposes.

- **Yields:** *str* – A string formatted as an SSE event, containing either the chunk data or an error message.

11.4 gateway.utils.usage module

11.4.1 get_token_count(messages: List[OpenAIMessage]) → int

Calculate the total number of tokens in a list of *OpenAIMessage* objects.

This function uses the *tiktoken* library to encode the content of the messages and counts the total number of tokens.

- **Parameters:** **messages** (*List* [*OpenAIMessage*]) – A list of *OpenAIMessage* DTOs, where each message contains a ‘content’ field that can be a string or an array of content-part dictionaries.
- **Returns:** An integer representing the total number of tokens in the messages.
- **Return type:** *int*

12 gateway

- gateway package
 - Subpackages
 - * gateway.app package
 - **create_app()**
 - Submodules
 - gateway.app.factory module
 - gateway.app.middleware module
 - * gateway.auth package
 - **validate_api_key()**
 - Submodules
 - gateway.auth.keys module
 - * gateway.clients package
 - **UpstreamClient**
 - Submodules
 - gateway.clients.upstream module

- * gateway.db package
 - db_key_check()
 - db_limit_change()
 - db_limit_check_and_allocation()
 - expire_quotas_by_batch()
 - get_quota_count()
 - refresh_quotas_by_batch()
 - Submodules
 - gateway.db.helpers module
 - gateway.db.keys module
 - gateway.db.limits module
 - gateway.db.refresh module
- * gateway.limits package
 - check_limits_costs()
 - Submodules
 - gateway.limits.quota module
- * gateway.logging package
 - prepare_usage_entry()
 - usage_logger()
 - Submodules
 - gateway.logging.extractor module
 - gateway.logging.metadata module
- * gateway.models package
 - OpenAIMessage
 - OpenAIRequest
 - UsageLogEntry
 - ValidatedRequest
 - Subpackages
- * gateway.routes package
 - Submodules
 - gateway.routes.chat module
 - gateway.routes.health module
- * gateway.utils package
 - calculate_cost()
 - get_token_count()
 - stream_generator()
 - Submodules
 - gateway.utils.price module
 - gateway.utils.streaming module
 - gateway.utils.usage module